# 第9章
# 类和对象

扫描全能王 创建

# 学习目标

- 类与对象的概念和定义；
- 类成员的访问权限；
- this指针的用法 。

扫描全能王 创建

# 基本知识

结构化程序设计（Structured Programming，SP）方法：

设计程序步骤：

- 根据实际问题的要求和对其进行分析，找出解决问题的方法和步骤（即设计算法）。
- 为了在计算机上具体实现出算法，需要为该种算法选择或构造适当的数据结构，通过对数据的操纵过程体现算法的思想。也就是说，程序是在数据的某种特定表示方式和结构的基础上对抽象算法的具体描述。
- 设计程序的结构为：




数据处理过程中，采用自顶向下、分而治之的方法，将整个程序按功能分为几个可独立编程的子过程模块，每个子模块完成指定的子任务，主过程通过调用各子过程来完成全部处理工作。

扫描全能王 创建

# 基本知识

- 结构化程序设计的特点：
  - 优点：
    - 该设计方法力求算法描述准确；
    - 对每一子过程模块容易进行程序正确性证明。
  - 缺点：
    - 该设计方法本质是面向“过程”的，而“过程”又是不稳定和多变的，因此不能直接反映人求解问题的思路；
    - 程序代码可重用性差；
    - 维护程序的一致性困难。

扫描全能王 创建

# 基本知识

- 面向对象的程序设计（Object-Oriented Programming，OOP）方法：
  - 直接以问题域（现实世界）中的事物为中心来思考和认识问题，并按照这些事物的本质特征把它们抽象为对象，以作为构成软件系统的基础，这样，在现实世界中有哪些值得注意的事物，在程序中就有哪些对象与之对应。由于程序与现实世界间有极强的对应关系，因此，程序可以用对象的概念很自然地进行思考，从而大大减少了软件开发的难度。

扫描全能王 创建

# 基本知识

- 对象（Object）：
  - 当利用计算机来处理客观世界的问题时，应首先对所要处理的客观事物进行数值化描述和状态模拟，这种对客观事物的数值化描述，就是所谓的对象。世界由各种对象组成，任何事物都是对象。
  - 每个对象都具有属性和方法这两方面的特征。属性描述了对象的状态和特征，方法说明了对象的行为和功能，并且对象的属性值通常只应由这个对象的方法来读取和修改，两者结合在一起就构成了对象的完整描述。

扫描全能王 创建

# 基本知识

- 对象的概念是面向对象技术的核心所在。面向对象技术中的对象就是现实世界中某个具体的物理实体在计算机逻辑中的映射和体现。
  - 一部移动电话，它是现实世界中的一个实体。它由天线、发射部件、接收部件、显示屏、按键、专用集成电路芯片及外壳组成；它有着其实在的功能，可以打电话，可以发短消息，可以存储、输入和编辑各种个人信息，甚至可以上网。这样一个实体可以在计算机世界中映射为一个计算机可以理解、可以操纵、具有前面所叙述的属性和操作的对象。

扫描全能王 创建

# 基本知识

- 一辆自行车，它由车架、车轮、脚踏和传动机构、变速机构等组成，它具有代步功能，它可以进行变速骑行，特别要强调的是它有一些特征可以把你的这辆自行车与其他自行车区分开来，其中最重要的是钢印号。这些都可以在面向对象的程序中用对象及其属性和操作模拟出来。

扫描全能王 创建

# 基本知识

- 类（Class）：
  - 为了便于理解和管理，人们习惯通过归类的方法从一个个具体对象中抽取出共同特征，以形成一般的概念。因此，具有相似属性和行为的一组对象，就称为类。
  - 类是对所有具有共同特征的事物的抽象，它是一个抽象的概念，相当与一个样板；而对象是一个类中的某个具体实体，是一个具体存在的、客观的实体，或者说是类的一个实例（instance），相当于是用样板制造出来的产品。

扫描全能王 创建

# 基本知识

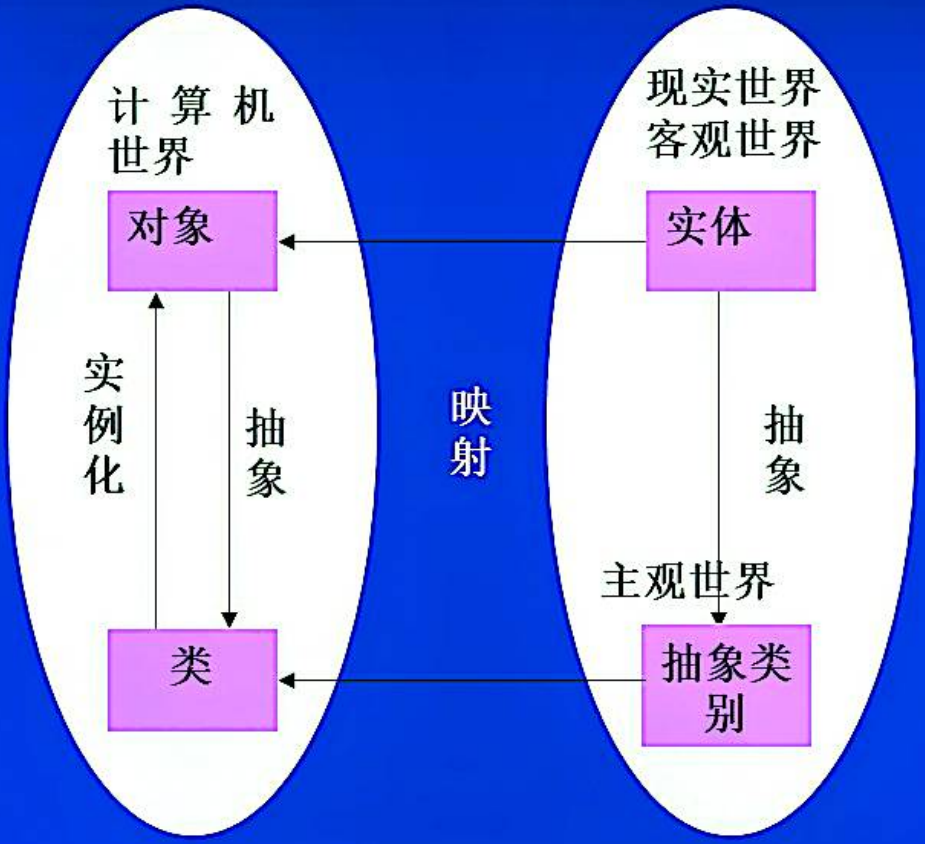



图 对象、实体与类

现实世界中的实体可以抽象出类别的概念。对应于计算机世界就有一个类（class）的概念，因为类是一个抽象的概念的对应体，所以计算机不给它分配内存，只给对象分配内存。图中表达了计算机世界与现实世界之间的对应关系。

扫描全能王 创建

# 基本知识

- 抽象(abstraction)是形成概念的必要手段，它是从许多事物中舍弃个别的、非本质性的特征，抽取共同及本质性的特征。
- 例如，谈到猫，世上没有任何两只猫是完全相同的，但是舍弃了每只猫相互之间的差异，把共同和本质性的特征抽取出来，就形成了“猫”这个概念。

扫描全能王 创建

# 基本知识

- 对于分析而言，抽象原则具有两方面的意义：
  - ① 尽管问题域中的事物很复杂，但分析员并不需要了解和描述它们的全部，只需要分析研究其中与系统目标有关的事物及其本质性特征。对于那些与系统目标无关的特征和许多具体的细节，即使有所了解，也应该舍弃。
  - ② 通过舍弃个体事物在细节上的差异，抽取其共同特征而得到一批事物的抽象概念。OOA中的类就是这样得到的。

扫描全能王 创建

# 基本知识

- 抽象是面向对象方法中使用最为广泛的原则，例如系统中的对象是对现实世界中事物的抽象；类是对象的抽象；数据成员是事物静态特征的抽象；成员函数是事物动态特征的抽象等。
- 在软件开发领域中，早在面向对象方法出现之前就已经开始运用抽象的原则，主要是过程抽象和数据抽象。

扫描全能王 创建

# 基本知识

- 封装（Class）：
  - 封装就是将数据成员和成员函数结合为一个不可分的系统单位，并只保留有限的对外接口，尽可能地隐蔽对象的内部细节。“数据隐藏”
  - 封装的信息隐蔽作用反映了事物的相对独立性。它实际上也是抽象原则的一种具体应用，它既体现了过程抽象，也体现了数据抽象。

扫描全能王 创建

# 基本知识

- 继承（Inheritance）：
  - 允许在已有类的基础上通过增加新特征而派生出新的类，使得相似的对象可以共享程序代码和数据。这样类构造成层次结构，一个类上层可有父类，下层可有子类。
  - 继承避免了对基类和派生类之间共同特征进行重复描述。
  - 继承简化了人们对事物的认识和描述，对软件复用是很有用的。

扫描全能王 创建

# 基本知识

- 消息（Message）：
  - 当一个对象需要另一个对象提供服务时，它向对方发出一个服务请求，而收到请求的对象会响应这个请求并完成指定的服务。这种向对象发出的服务请求就称为消息。
- 多态性（Polymorphism）：
  - 不同对象可以用同一个方法名字来接受外界的消息，但可用不同的行为来实现该动作。

扫描全能王 创建

# 9.1 类

- 类的定义格式：

```
class 标识符{
 private:
    私有成员说明;
 public:
    公有成员说明;
 protected:
    保护成员说明;
};
```

```
using namespace std;
#include <string>
#include <iostream>
class  Person{
  char name[20];
  char sex;
  char id_no[30];
public:
  void SetName(char *);
  void Getname(char *);
};
Person:: SetName(char *p)
 {strcpy(name,p);}
Person:: Getname(char *p)
 {strcpy(p,name);}
```

扫描全能王 创建

# 9.1 类

1. 缺省是私有说明，所以private在大部分情形下省去
2. protected 部分不一定有
3. 三部分的次序可随意，但private只有在第一部分才可省略。
4. 成员指c++中所允许的所有数据类型或构造类型或成员函数。
5. 对成员函数而言，在类中可只给出原形（特别在函数复杂时），在类外以

   类型名：：成员函数（参数表）

   { 函数体；}

   给出，一般，数据说明为私有，成员函数说明为公有，外部不能直接访问数据，而只能通过成员函数存取修改数据，即通过成员函数向外界提供接口。
6. 成员函数的重载：

   类中成员函数与普通函数一样，成员函数可以带有缺省的参数，也可以重载函数。

扫描全能王 创建

# 9.1 类

- 定义一个类时，其成员函数的函数体的定义可以在定义类的类体中直接定义。

```
using namespace std;
#include <string>
#include <iostream>
class  Person{
  char name[20];
  char sex;
  char id_no[30];
public:
  void SetName(char *);
  void Getname(char *);
};
Person:: SetName(char *p)
 {strcpy(name,p);}
Person:: Getname(char *p)
 {strcpy(p,name);}
```

```
using namespace std;
#include <string>
#include <iostream>
class  Person{
  char name[20];
  char sex;
  char id_no[30];
public:
  void SetName(char *p)
      {strcpy(name,p);}
  void Getname(char *p)
      {strcpy(p,name);}
```

```
inline void Person::SetName(char* p)
```

扫描全能王 创建

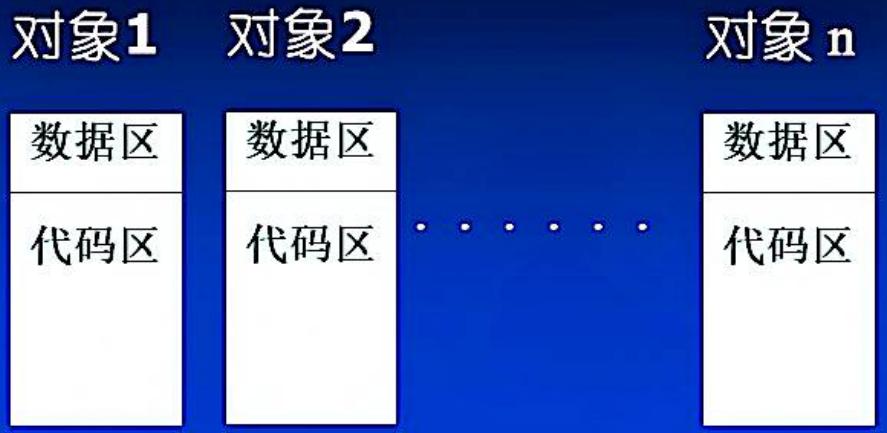



图1 各对象完全独立地安排内存的方案

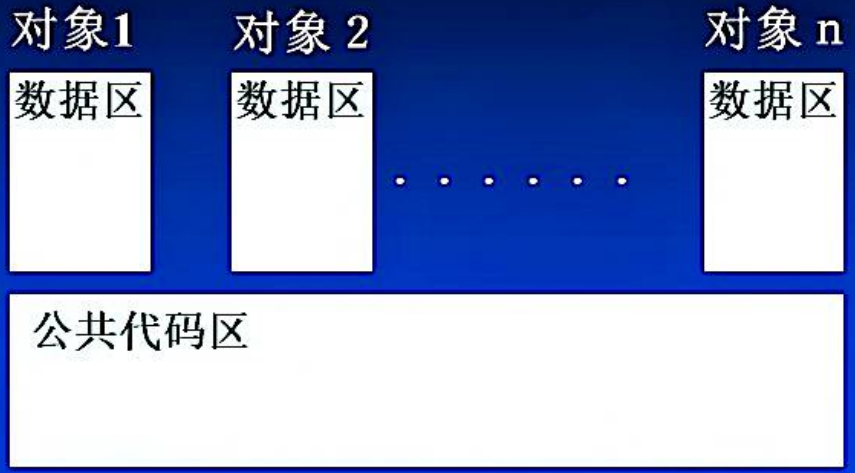



图2 各对象的代码区共用的方案

图1是系统为每一个对象分配了全套的内存，包括安放成员数据的数据区和安放成员函数的代码区。但区别同一个类的各个不同的对象的属性是由数据成员决定的，不同对象的数据成员的内容是不一样的；而行为（操作）是用函数来描述的，这些操作的代码对所有的对象都是一样的。

图2仅为每个对象分配一个数据区，代码区（放成员函数的区域）为各对象类共用。

图1对应的是在类说明中定义函数，而图2对应的是在类说明外部定义函数。

扫描全能王 创建

# 9.1 类

- 类和结构体类型：
  1. 结构体类型与类雷同，结构体中的成员也可以是数据成员或函数成员，也可使用关键字private、 public和protected限定访问权限；
  2. 结构体类型是类的一个特例；
  3. 但结构体类型中成员的缺省存取权限是共有的；
  4. 通常是在仅需要描述数据结构时使用结构体，既要描述数据又要描述处理方法时，使用类。

扫描全能王 创建

# 9.2 对象

- 对象就是具有类类型的变量。
- 三种说明对象的方法：
  - 先定义类，再说明对象；
  - 定义类的同时说明对象；
  - 直接说明对象，不定义类的类名。
- 当类中成员数据访问权限指定为私有或受保护时，定义对象时不允许对它的成员数据进行初始化。

扫描全能王 创建

# 9.2 对象

- 对象的使用：
  - 用成员选择运算符“.”来访问对象成员，用指向对象的指针来访问对象成员时利用“->”；
  - 同类型对象之间可以整体赋值；
  - 对象用作函数的参数时，属于赋值调用；函数可以返回一个对象；
  - 可定义类类型的指针、类类型的引用、对象数组、指向类类型的指针数组和指向一维或多维数组的指针变量；
  - 一个类的对象可作为另一个类的成员。

扫描全能王 创建

```
using namespace std;
#include <string>
#include <iostream>
class  Person{
  char name[20];
  char sex;
  char id_no[30];
public:
  void SetName(char *p)
  {  strcpy(name,p);}
  void Getname(char *p)
  {  strcpy(p,name);}
  void Setid_no(char *);
  void  Getid_no(char *);
};

Person::Setid_no(char *p)
   { strcpy(id_no,p);}

Person::Getid_no(char *p)
   { strcpy(p,id_no);}
```

类定义

```
 int main()
{
 Person a,b;
  char s[30];
  a.Setid_no("20110001-01");
  a.SetName("wang");
  a.Getname(s);
  cout<<s<<endl;
  b=a;
  b.Getid_no(s);
  cout<<s<<endl;
  a.sex='m';    //error
 }
```

我看不见数据



公有部分

SetName()
GetName()
Setid_no()
Getid_no()

私有部分

Person对象

Person类的
对象a,b定义

扫描全能王 创建

```
using namespace std;
#include <string>
#include <iostream>
class  Person{
    char name[20];
    char id_no[30];
public:
    char sex;
    void SetName(char *);
    void Getname(char *);
    void Setid_no(char *);
    void  Getid_no(char *);
};
int main()
{
  a.sex='m';
}
```




扫描全能王 创建

- 同类型对象之间可以整体赋值

```
using namespace std;
#include <iostream>
class Complex{
  double Real,Imag;
  public:
  void SetCom(double a,double b)
     { Real=a; Imag=b;}
  void PrintCom()
     { cout<<Real<<' '<<Imag<<endl;}
  Complex AddCom(Complex x)
     { Complex t;
       t.Real=Real+x.Real;
       t.Imag=Imag+x.Imag;
       return t;}
};
```

```
int main()
{ Complex a,b,c;
   a.SetCom(1.0,2.0);
   b.SetCom(2.0,3.0);
   c=a.AddCom(b);
   c.PrintCom();
   c=a;
   c.PrintCom();
}
```

扫描全能王 创建

# 类作用域和对象的作用域

- 定义类时，用一对花括号括起来的区域为类的作用域；
- 对象的作用域与变量的作用域完全相同。
  - 全局对象：作用域是整个程序；
  - 局部对象：作用域是定义它的函数体。

```
class  Desk{
  int weight;
  int high;
  int width;
  int length;
  void Show();
};
Desk a;//定义全局对象
void fn()
{
    high=100;//error
    Show();//error
    Desk b;//定义局部对象
}
```

扫描全能王 创建

关于类与对象,下面哪一种说法是错误的?

A 一个对象是某个类的一个实例。

B 一个实例是某个类型经实例化所产生的一个实体或值。

C 创建一个对象必须指定被实例化的一个类。

D 一个类的多个对象之间不仅持有独立的数据成员,而且成员函数也是独立的。

扫描全能王 创建

关于类的数据成员,下面哪一种说法是错误的?

A 假设一个类名为A,那么“A a;”不能作为类A 的数据成员。

B 在说明一个非静态数据成员时,可以添加初始化。

C 类中的多个数据成员变量不能重名。

D 如果有两个数据成员的可见性不同,它们就可以重名。

扫描全能王 创建

关于类的成员的可见性，下面说法中错误的是：

A 私有成员只能在本类中访问，而不能被类外代码访问

B 一般将类的数据成员说明为私有成员，但不是绝对的

C 公有成员能被类外代码访问，而不能被同一类中的代码访问

D 一般将类的成员函数说明为公有成员，但不是绝对的

扫描全能王 创建

关于对象成员的访问,下面哪一种说法是错误的?

A 对于一个对象,可用“.”运算符来访问其成员。

B 对于一个对象引用,可用“->”运算符来访问其成员。

C 如果被访问成员是公有的,访问表达式可出现在main 函数中。

D 如果被访问成员是私有的,访问表达式只能出现在本类中。

扫描全能王 创建

关于类的成员函数,下面哪一种说法是错误的?

A 一般来说,一个类的成员函数对本类中的数据成员进行读写计算。

B 如果一个数据成员希望是只读的,该成员应说明为私有,而且用一个公有的getXxx 成员函数来读取它的值。

C 一个类中的一组成员函数不能重名。

D 公有成员函数不应返回本类的私有成员的指针或引用。

扫描全能王 创建

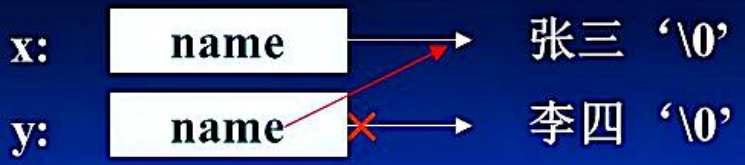
# 9.3 this 指针

- 当对象调用一个成员函数时，成员函数除接受实参外还接受了该对象的地址，这个地址被一个隐含的形参this指针所获取。
- this指针只允许使用，不允许改变其值。

```
using namespace std;
#include <iostream>
class Complex{
  double Real,Imag;
  public:
  void SetCom(double a,double b)
     { Real=a; // this->Real=a;
       Imag=b; // this->Imag=b;
     }
};
int main()
{ Complex a;
   a.SetCom(1.0,2.0);
}
```

扫描全能王 创建

```
using namespace std;
#include <string>
#include <iostream>
class Person{
  char *name;
public:
  void SetName(char *s)
  { name=new char [strlen(s)+
  void GetName(char *s)
   { strcpy(s,name);}
 void Delete(){delete name;}
};
int main()
{
   Person x,y;
   char s[20];
   x.SetName("张三");
   y.SetName("李四");
   x.GetName(s);
   cout<<"x:"<<s<<endl;
   y=x;
   y.GetName(s);
   cout<<"y:"<<s<<endl;
   x.Delete();
   y.Delete();
}
```




加入：

```
void Copy(Person & s)
{ if(name!=NULL) delete name;
   name= new char[strlen(s.name)+1];
   strcpy(name,s.name);}
```

改为：y.Copy(x)

x:张三
y:李四

如果：x.Copy(x)

注意引用问题

```
void Copy(Person &s)
{
  if(this==& s) return ;
  if(name!=NULL) delete name;
  name= new char[strlen(s.name)+1];
  strcpy(name,s.name);}
```

扫描全能王 创建

关于this指针，下面说法中错误的是：

A 每个非静态成员函数都隐含一个this指针

B this指针在成员函数中始终指向当前作用对象

C 在成员函数中直接访问成员m，隐含着this->m

D 在使用this指针之前，应该显式说明this

扫描全能王 创建

关于类的成员,下面哪一种说法是错误的?

A 类中的一个非静态数据成员表示该类每个对象都持有的一个值。

B 调用类中的一个非静态成员函数必须确定一个作用对象。

C 类中至少应包含一个成员。

D 类中的各个成员的说明没有严格次序。

扫描全能王 创建

# 魔兽争霸DOTA简化版

- 英雄
- 技能

扫描全能王 创建